
Service object in Kubernetes (Cluster-IP and NodePort)

Rohit Surwade

CONTENT

[Cluster IP](#)

[Manifest to create one Deployment object.](#)

[How to ping the container inside the POD using POD's ip](#)

[Creating a Service Object \(Cluster IP\)](#)

[How to access the container using the Cluster IP.](#)

[Testing the working of Cluster IP after deleting the POD](#)

[NodePort](#)

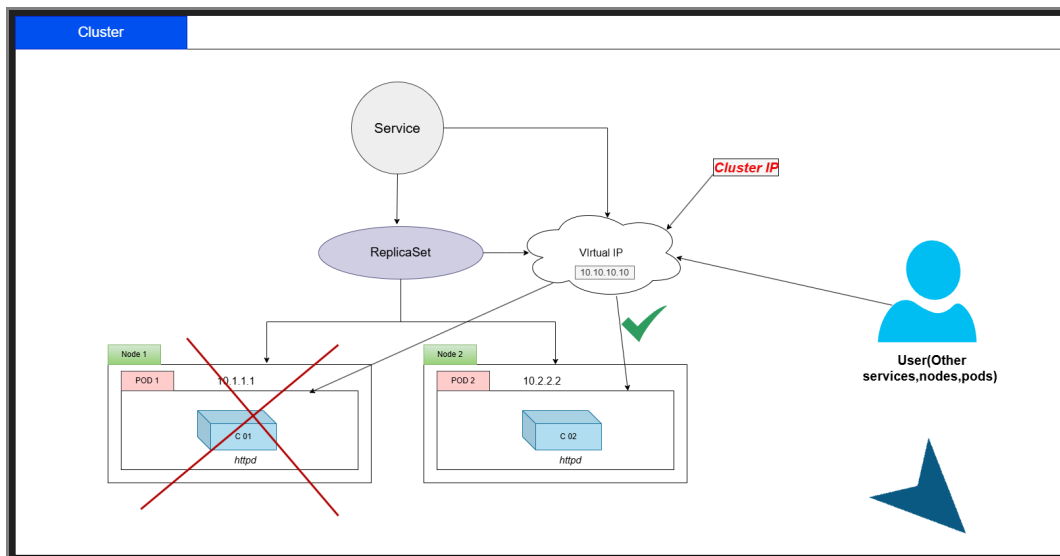
[A manifest to create a Deployment object](#)

[Creating a Service object \(NodePort\)](#)

[Applying the manifests to create the respective objects](#)

[Testing the the functionality of NodePort](#)

Cluster IP



1.

ClusterIP is the default type of Service that provides internal access to the services running in the cluster.

A ClusterIP service exposes the service on a virtual IP address (VIP) that is only reachable from within the Kubernetes cluster.

This means that any Pod within the cluster can access the service, but it cannot be accessed from outside the cluster unless additional configurations are made.

Manifest to create one Deployment object.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mydeployments
spec:
  replicas: 1
  selector:
    matchLabels:
      name: deployment
  template:
    metadata:
      name: testpod1
      labels:
        name: deployment
    spec:
      containers:
        - name: c00
          image: httpd
          ports:
            - containerPort: 80
```

2.

(Kubernetes will create a Deployment named mydeployments. The Deployment will create one Pod (since replicas: 1), which contains one container that runs the httpd image (Apache HTTP Server). The container will expose port 80, meaning it will be ready to serve HTTP traffic. Kubernetes will ensure that if the Pod fails (e.g., crashes or is deleted), the Deployment will automatically create a new Pod to maintain the desired state (i.e., 1 running Pod). This YAML is a simple example of using a Deployment to manage Pods, and it can be expanded to handle more complex scenarios, such as multi-container Pods, volume mounts, environment variables, and other configurations.)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mydeployments
spec:
  replicas: 1
  selector: # tells the controller which pods to watch/belong to
  matchLabels:
    name: deployment
  template:
    metadata:
      name: testpod1
    labels:
      name: deployment
    spec:
      containers:
        - name: c00
          image: httpd
          ports:
            - containerPort: 80
```

How to ping the container inside the POD using POD's ip

```

root@ip-172-31-36-215:/home/ubuntu# D deployhttpd.yml
root@ip-172-31-36-215:/home/ubuntu# kubectl apply -f deployhttpd.yml
deployment.apps/mydeployments created
root@ip-172-31-36-215:/home/ubuntu# kubectl get pods
A
NAME                                READY   STATUS    RESTARTS   AGE
mydeployments-59fdd57666-d85dd      1/1     Running   0           5s
root@ip-172-31-36-215:/home/ubuntu# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   B
mydeployments-59fdd57666-d85dd      1/1     Running   0           11s   10.244.0.56
root@ip-172-31-36-215:/home/ubuntu# minikube ssh curl 10.244.0.56:80
C
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-36-215:/home/ubuntu#

```

3.

commands:

1. kubectl apply -f yaml-filename.yml
2. kubectl get pods -o wide
3. minikube ssh curl POD's-IP:Port no

NOTE:

After applying the manifest our deployment object is created it has a pod(testpod1) and a container(c 00) within that pod with httpd image running on it.

Now, we can access it by using POD's IP but that is not a good practice cause PODS are ephemeral in Nature and their IP's will keep changing.
so to fix that we will be creating a SERVICE Object that is Cluster IP.

Creating a Service Object (Cluster IP)

```

root@ip-172-31-36-215:~/demo# cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: demoservice
spec:
  ports:
  - port: 80
    targetPort: 80
  selector:
    name: deployment
  type: ClusterIP
# Defines to create Service type Object
# Containers port exposed
# Pods port
# Apply this service to any pods which has the specific label
# Specifies the service type i.e ClusterIP or NodePort

```

4.

SERVICE(CLUSTER IP)

(This Service will create a network endpoint inside the cluster that forwards traffic from port 80 to the Pods with the label name: deployment on port 80. ClusterIP means that the Service is accessible only within the Kubernetes cluster, not externally.

Example Scenario:

If you have a Deployment with Pods labeled name: deployment (as in your earlier manifest), this Service will allow other Pods inside the cluster to communicate with those Pods on port 80.

For example, other Pods or services in the cluster can access the demoservice using demoservice:80 (Kubernetes handles DNS resolution).)

```

apiVersion: v1
kind: Service # Defines to create Service type Object
metadata:
  name: demoservice
spec:
  ports:
  - port: 80 # Containers port exposed
    targetPort: 80 # Pods port
  selector:
    name: deployment # Apply this service to any pods which has the specific label
  type: ClusterIP # Specifies the service type i.e ClusterIP or NodePort

```

How to access the container using the Cluster IP.

```

root@ip-172-31-36-215:/home/ubuntu# minikube ssh curl 10.244.0.56:80
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-36-215:/home/ubuntu# vi service.yml
root@ip-172-31-36-215:/home/ubuntu# kubectl apply -f service.yml
service/demoservice created
root@ip-172-31-36-215:/home/ubuntu# kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
demoservice  ClusterIP   10.96.135.76  <none>        80/TCP     11s
kubernetes   ClusterIP   10.96.0.1     <none>        443/TCP    5d18h
root@ip-172-31-36-215:/home/ubuntu# curl 10.96.135.76:80
^L^X^C
root@ip-172-31-36-215:/home/ubuntu# minikube ssh curl 10.96.135.76:80
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-36-215:/home/ubuntu#

```

5.

commands:

1. kubectl get svc
2. minikube ssh curl CLUSTER-IP:port no

Testing the working of Cluster IP after deleting the POD

```

root@ip-172-31-36-215:/home/ubuntu# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   NODE       NOMINATED NODE   READINESS G
mydeployments-59fdd57666-d85dd  1/1     Running   0           6m24s  ip-172-31-36-215  <none>           <none>
root@ip-172-31-36-215:/home/ubuntu# kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
demoservice  ClusterIP   10.96.135.76  <none>        80/TCP     2m20s
kubernetes   ClusterIP   10.96.0.1     <none>        443/TCP    5d18h
root@ip-172-31-36-215:/home/ubuntu# minikube ssh curl 10.96.135.76:80
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-36-215:/home/ubuntu# kubectl delete pod mydeployments-59fdd57666-d85dd
pod "mydeployments-59fdd57666-d85dd" deleted
root@ip-172-31-36-215:/home/ubuntu# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   NODE       NOMINATED NODE   READINESS G
mydeployments-59fdd57666-tnv81  1/1     Running   0           9s    ip-172-31-36-215  <none>           <none>
root@ip-172-31-36-215:/home/ubuntu# minikube ssh curl 10.96.135.76:80
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-36-215:/home/ubuntu#

```

6.

commands:

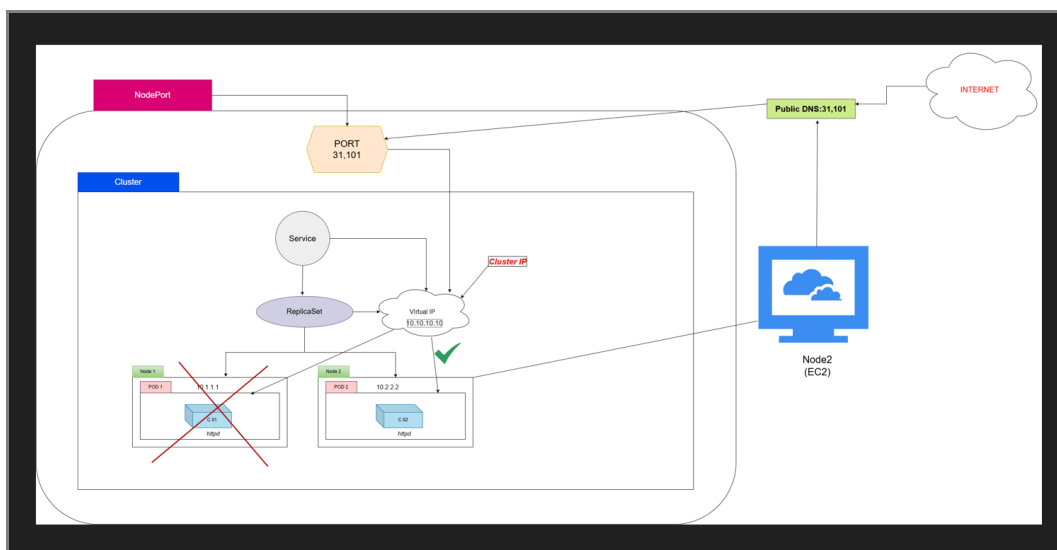
1. kubectl get pods -o wide
2. kubectl get svc
3. minikube ssh curl CLUSTER-IP:port no
4. kubectl delete pod pod-name

NOTE:

EVEN AFTER DELETING THE POD WE ARE STILL ABLE TO ACCESS THE IT USING THE SAME CLUSTER IP. BECAUSE Pods that match the selector will have their traffic forwarded by the ClusterIP service. Other Pods within the cluster can communicate with clusterip-service by sending requests to port 80.

Kubernetes will automatically load-balance the traffic across all Pods that match the selector forwarding requests to port 80 of those Pods.

NodePort



7.

NodePort is a type of Kubernetes Service that exposes a pod or set of pods to external traffic by assigning a static port on each node in the cluster.

This allows external clients to access the service using the node's IP address and a specific port (the NodePort).

NOTE:

- SINCE WE ARE USING “DOCKER” AS “VM-DRIVER” WE WON'T BE ABLE TO ACCESS CONTAINER USING PUBLIC DNS OF THE EC2 INSTANCE.
- WE WILL BE ABLE TO ACCESS IT USING THE NODE'S-IP:NODEPORT.

A manifest to create a Deployment object


```
root@ip-172-31-36-215:/home/ubuntu
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mydeployments
spec:
  replicas: 1
  selector: # tells the controller which pods to watch/belong to
    matchLabels:
      name: deployment
  template:
    metadata:
      name: testpod1
    labels:
      name: deployment
    spec:
      containers:
        - name: c00
          image: nginx
          ports:
            - containerPort: 80
```

8.

This manifest describes a Kubernetes Deployment named mydeployments that ensures one replica of a pod is running. The pod runs a container named c00 using the nginx image and exposes port 80. The deployment manages pods with the label name: deployment and ensures that the desired state (one running pod) is always maintained.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mydeployments
spec:
  replicas: 1 # Number of replicas (pods) to maintain
  selector: # Specifies the labels that identify the pods managed by this
  deployment
  matchLabels:
    name: deployment
  template: # Defines the pod template that is used to create the pods
  metadata:
    labels:
      name: deployment # This label should match the selector's matchLabels
  spec:
    containers:
      - name: c00 # Container name
        image: nginx # Image for the container
        ports:
          - containerPort: 80 # Port exposed by the container
```

Creating a Service object (NodePort)

```
root@ip-172-31-36-215:~/home/ubuntu
apiVersion: v1
kind: Service
metadata:
  name: demoservice
spec:
  ports:
    - port: 80
      targetPort: 80
  selector:
    name: deployment
  type: NodePort
# Defines to create Service type Object
# Containers port exposed
# Pods port
# Apply this service to any pods which has the specific label
# Specifies the service type i.e ClusterIP or NodePort
```

9.

- **demoservice** is a Service of type NodePort, which exposes the pods it targets (those with the label name: deployment) on port 80.
- The service listens on port 80, and any traffic to this port will be forwarded to the targetPort of the pods (also port 80).
- The NodePort type means that the service is accessible externally on any node's IP, with an automatically assigned port from the 30000-32767 range.

This service allows external access to the pods running behind it by routing traffic through the NodePort, making the service accessible from outside the Kubernetes cluster, such as through a browser or curl command on the assigned port.

apiVersion: v1

kind: Service

metadata:

name: demoservice

spec:

ports:

- port: 80 # Port exposed by the service

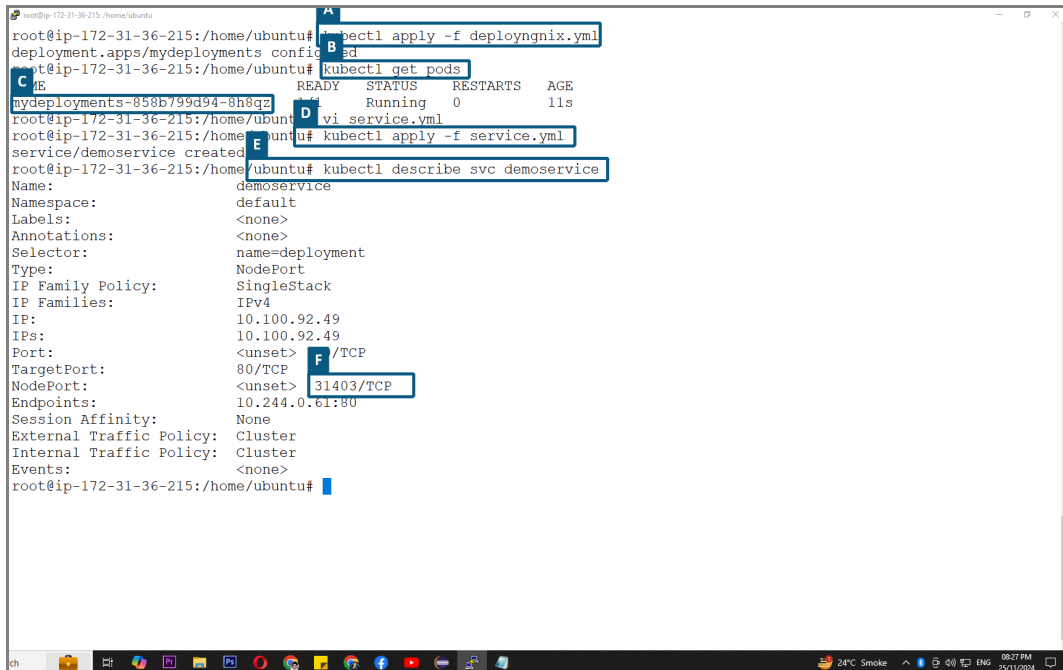
targetPort: 80 # Port on the pods that the service forwards traffic to

selector:

name: deployment # Pods with this label will be targeted by the service

type: NodePort # Exposes the service on a port on each node

Applying the manifests to create the respective objects



```
root@ip-172-31-36-215:/home/ubuntu# kubectl apply -f deploynginx.yml
deployment.apps/mydeployments configured
root@ip-172-31-36-215:/home/ubuntu# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mydeployments-858b799d94-8h8g2     1/1     Running   0           11s
root@ip-172-31-36-215:/home/ubuntu# vi service.yml
root@ip-172-31-36-215:/home/ubuntu# kubectl apply -f service.yml
service/demoservice created
root@ip-172-31-36-215:/home/ubuntu# kubectl describe svc demoservice
Name:                             demoservice
Namespace:                         default
Labels:                             <none>
Annotations:                         <none>
Selector:                           name=deployment
Type:                               NodePort
IP Family Policy:                   SingleStack
IP Families:                        IPv4
IP:                                 10.100.92.49
IPs:                                 10.100.92.49
Port:                               <unset> /TCP
TargetPort:                         80/TCP
NodePort:                           <unset> 31403/TCP
Endpoints:                          10.244.0.61:80
Session Affinity:                   None
External Traffic Policy:            Cluster
Internal Traffic Policy:            Cluster
Events:                             <none>
root@ip-172-31-36-215:/home/ubuntu#
```

10.

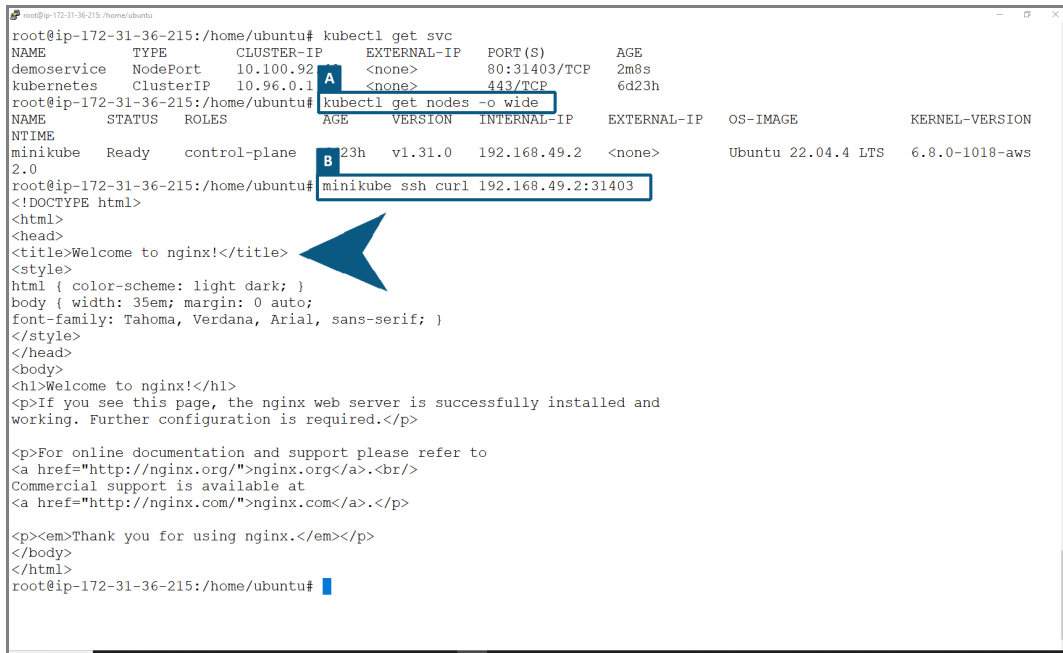
commands:

1. `kubectl apply -f yaml-file-name.yml`
2. `kubectl get pods`
3. `kubectl describe svc service-name`

NOTE:

- We have created a Deployment object which has one container(c 00) with nginx image running on it.
- NodePort has given us a port(An automatically assigned port from the 30000-32767 range) that is “31403”.

Testing the the functionality of NodePort



The screenshot shows a terminal window with the following commands and output:

```
root@ip-172-31-36-215:/home/ubuntu# kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
demoservice         NodePort    10.100.92     <none>         80:31403/TCP    2m8s
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP         6d23h
```

Then, the command `kubectl get nodes -o wide` is executed, showing the node details:

```
root@ip-172-31-36-215:/home/ubuntu# kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE     VERSION   INTERNAL-IP    EXTERNAL-IP    OS-IMAGE             KERNEL-VERSION
minikube            Ready     control-plane  23h     v1.31.0   192.168.49.2   <none>         Ubuntu 22.04.4 LTS   6.8.0-1018-aws
```

Finally, the command `minikube ssh curl 192.168.49.2:31403` is executed, resulting in the following HTML output:

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

11.

commands:

1. `kubectl get nodes -o wide`
2. `minikube ssh curl node's-ip:NodePort`

NOTE:

- SINCE WE ARE USING “DOCKER” AS “VM-DRIVER” WE WON'T BE ABLE TO ACCESS CONTAINER USING PUBLIC DNS OF THE EC2 INSTANCE.
- WE WILL BE ABLE TO ACCESS IT USING THE NODE'S-IP:NODEPORT.